

---

# DATA STRUCTURES - COMPLETE TEACHING PRESENTATION

Topics: Linked List | Queue | Stack | Hashing

---

Copy each SLIDE section into your PowerPoint/Google Slides

---

## SLIDE 1: TITLE SLIDE

DATA STRUCTURES Mastering Linked Lists, Stacks, Queues & Hashing

A Complete Guide with Code, Visualizations & Applications

[Your Name] [Date]

## SLIDE 2: AGENDA

Today's Agenda

1. Linked List • Singly Linked List • Doubly Linked List • Circular Linked List • Operations & Complexity
2. Stack • LIFO Principle • Array & Linked List Implementation • Applications
3. Queue • FIFO Principle • Types of Queues • Implementation & Applications
4. Hashing • Hash Functions • Collision Resolution • Hash Tables in Practice

## SLIDE 3: INTRODUCTION TO DATA STRUCTURES

What are Data Structures?

Definition: A data structure is a specialized format for organizing, processing, retrieving and storing data to suit a specific purpose.

Why Data Structures Matter:

- Efficiency → Faster algorithms, less memory
- Scalability → Handle growing data gracefully
- Reusability → Standard solutions to common problems
- Abstraction → Hide complexity, expose functionality

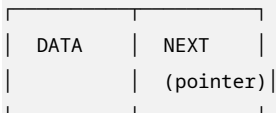
Classification: Linear: Non-Linear:  
Linked List

Array Tree Stack Graph Queue

## SLIDE 4: LINKED LIST - INTRODUCTION

LINKED LIST

Definition: A linear data structure where elements (nodes) are connected using pointers. Each node contains:



Why Linked List over Array?

|                 |                     | Operation        |            | Array              | Linked List      |        |         |
|-----------------|---------------------|------------------|------------|--------------------|------------------|--------|---------|
| Insertion       | $O(n)$ - shift req  | $O(1)$ - at head | Deletion   | $O(n)$ - shift req | $O(1)$ - at head | Access |         |
| $O(1)$ - random | $O(n)$ - sequential | Memory           | Contiguous | Dynamic/Scattered  | Size             | Fixed  | Dynamic |

## SLIDE 5: SINGLY LINKED LIST

SINGLY LINKED LIST

Structure: Each node points to the next node only.



Node 1 Node 2 Node 3 Node 4

10 20 30 40 NULL

Node Structure (Python):

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

Linked List Class:

```
class LinkedList:
    def __init__(self):
        self.head = None
```

# SLIDE 6: SINGLY LINKED LIST - INSERTION

## INSERTION IN SINGLY LINKED LIST

### 1. INSERT AT BEGINNING (O(1))

Before: HEAD 20 NULL

Step 1: Create new node 10 NULL ← new\_node

Step 2: new\_node.next = head 10 20 NULL

Step 3: head = new\_node HEAD 10 20 NULL

Code:

```
def insert_at_beginning(self, data):
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node
```

## SLIDE 7: SINGLY LINKED LIST - INSERTION (CONT.)

### 2. INSERT AT END ( $O(n)$ )

Traverse to last node, then append.

```
def insert_at_end(self, data):
    new_node = Node(data)
    if not self.head:
        self.head = new_node
        return

    current = self.head
    while current.next:          # Traverse to end
        current = current.next

    current.next = new_node      # Link last node to new
```

### 3. INSERT AT POSITION ( $O(n)$ )

```
def insert_at_position(self, data, position):
    if position == 0:
        self.insert_at_beginning(data)
        return

    new_node = Node(data)
    current = self.head

    for _ in range(position - 1):
        if not current:
            raise IndexError("Position out of bounds")
        current = current.next

    new_node.next = current.next
    current.next = new_node
```

## SLIDE 8: SINGLY LINKED LIST - DELETION

### DELETION IN SINGLY LINKED LIST

#### 1. DELETE FROM BEGINNING ( $O(1)$ )

Before: HEAD                      10            20            ...

After: HEAD = head.next

20

...

```
def delete_from_beginning(self):
    if not self.head:
        return None
    data = self.head.data
    self.head = self.head.next
    return data
```

## 2. DELETE FROM END ( $O(n)$ )

```
def delete_from_end(self):
    if not self.head:
        return None
    if not self.head.next:
        data = self.head.data
        self.head = None
        return data

    current = self.head
    while current.next.next:      # Stop at second-last
        current = current.next

    data = current.next.data
    current.next = None
    return data
```

# SLIDE 9: SINGLY LINKED LIST - TRAVERSAL & SEARCH

## TRAVERSAL & SEARCH

Traversal ( $O(n)$ ):

```
def display(self):
    current = self.head
    while current:
        print(current.data, end=" → ")
        current = current.next
    print("NULL")
```

Search ( $O(n)$ ):

```
def search(self, key):
    current = self.head
    position = 0
    while current:
        if current.data == key:
```

```

        return position
    current = current.next
    position += 1
    return -1 # Not found

```

Reverse a Linked List ( $O(n)$ ):

```

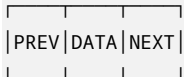
def reverse(self):
    prev = None
    current = self.head
    while current:
        next_node = current.next
        current.next = prev
        prev = current
        current = next_node
    self.head = prev

```

## SLIDE 10: DOUBLY LINKED LIST

### DOUBLY LINKED LIST

Each node has two pointers: previous and next.



Structure:

|      |      |      |    |    |    |
|------|------|------|----|----|----|
| NULL |      | NULL | 10 | 20 | 30 |
|      | HEAD |      |    |    |    |

Node Structure:

```

class DNode:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

```

Advantages: Bidirectional traversal Easy deletion ( $O(1)$ ) if node is known Can move backward easily

Disadvantages: Extra memory for prev pointer More complex implementation

## SLIDE 11: DOUBLY LINKED LIST - INSERTION

### DOUBLY LINKED LIST - INSERTION AT BEGINNING

|              |    |      |
|--------------|----|------|
| Before: NULL | 20 | NULL |
|--------------|----|------|

Step 1: Create new node                  NULL 10 NULL  $\leftarrow$  new\_node

|                              |      |    |    |      |
|------------------------------|------|----|----|------|
| Step 2: new_node.next = head | NULL | 10 | 20 | NULL |
|------------------------------|------|----|----|------|

|                                               |      |    |    |      |
|-----------------------------------------------|------|----|----|------|
| Step 3: head.prev = new_node, head = new_node | NULL | 10 | 20 | NULL |
| HEAD                                          |      |    |    |      |

```
def insert_at_beginning(self, data):
    new_node = DNode(data)
    if not self.head:
        self.head = new_node
    return
    new_node.next = self.head
    self.head.prev = new_node
    self.head = new_node
```

## SLIDE 12: CIRCULAR LINKED LIST

## CIRCULAR LINKED LIST

Last node points back to the first node.

|                  |    |    |    |
|------------------|----|----|----|
| Singly Circular: | 10 | 20 | 30 |
| HEAD             |    |    |    |

|                  |    |      |
|------------------|----|------|
| Doubly Circular: | 10 | 20   |
| 30               |    | HEAD |

Use Cases: • Round-robin scheduling (CPU) • Music playlist (repeat mode) • Multiplayer game turns

## SLIDE 13: LINKED LIST COMPLEXITY SUMMARY

### LINKED LIST TIME COMPLEXITY

|                   | Operation      |          |                |             |                  |             |                   |                |                |        |
|-------------------|----------------|----------|----------------|-------------|------------------|-------------|-------------------|----------------|----------------|--------|
|                   | SLL            | DLL      | Array          |             |                  |             |                   |                |                |        |
| Access (by index) | $O(n)$         | $O(n)$   | $O(1)$         | Search      | $O(n)$           | $O(n)$      | $O(n)$            | Insert at Head | $O(1)$         | $O(1)$ |
| $O(n)$            | Insert at Tail | $O(n)^*$ | $O(n)^*$       | $O(1)^{**}$ | Insert at Middle | $O(n)$      | $O(n)$            | $O(n)$         | Delete at Head | $O(1)$ |
| $O(1)$            | $O(1)$         | $O(n)$   | Delete at Tail | $O(n)$      | $O(1)$           | $O(1)^{**}$ | Delete Given Node | $O(n)$         | $O(1)$         | $O(n)$ |

- $O(1)$  with tail pointer  $^{**}$  Amortized,  $O(n)$  if resize needed

Space Complexity:  $O(n)$  for all

## SLIDE 14: LINKED LIST APPLICATIONS

### REAL-WORLD APPLICATIONS OF LINKED LISTS

1. Operating Systems • Process scheduling queues • Memory management (free blocks) • File allocation tables
2. Web Browsers • Forward/Backward navigation (DLL) • Tab management
3. Music Players • Playlist with next/previous (DLL) • Repeat/Shuffle modes (Circular)
4. Image Viewers • Slideshow with prev/next
5. Blockchain • Each block linked to previous hash
6. Undo/Redo Functionality • Stack of states using linked list
7. Polynomial Representation • Each term as a node (coefficient, exponent)

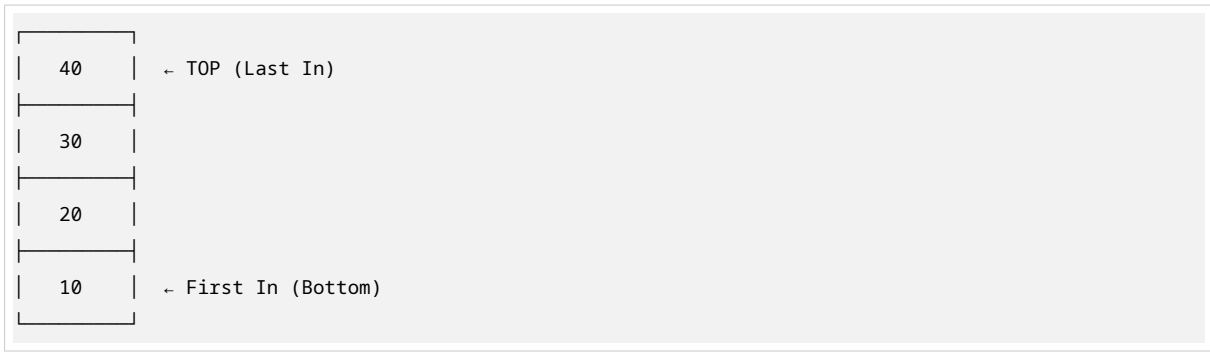
## SLIDE 15: STACK - INTRODUCTION

### STACK

Definition: A linear data structure that follows LIFO (Last In, First Out) principle.

Visual Representation:





Real-world Analogy: • Stack of plates • Stack of books • Browser back button • Undo in text editors

| Key Operations: |                                   | Operation | Description                   |
|-----------------|-----------------------------------|-----------|-------------------------------|
| PUSH            | Add element to top of stack       | POP       | Remove and return top element |
| PEEK/TOP        | View top element without removing | isEmpty   | Check if stack is empty       |
| SIZE            | Return number of elements         |           |                               |

## SLIDE 16: STACK - OPERATIONS VISUALIZATION

### STACK OPERATIONS - STEP BY STEP

Initial: Empty Stack

|                                  |    |       |    |    |
|----------------------------------|----|-------|----|----|
| PUSH(10):                        | 10 | ← TOP |    |    |
| PUSH(20):                        | 20 | ← TOP | 10 |    |
| PUSH(30):                        | 30 | ← TOP | 20 | 10 |
| POP() → Returns 30:              | 20 | ← TOP | 10 |    |
| PEEK() → Returns 20 (no removal) |    |       |    |    |

## SLIDE 17: STACK - ARRAY IMPLEMENTATION

### STACK IMPLEMENTATION USING ARRAY

```
class StackArray:
    def __init__(self, capacity=100):
        self.stack = []
        self.capacity = capacity

    def push(self, item):
```

```

    if len(self.stack) >= self.capacity:
        raise OverflowError("Stack is full")
    self.stack.append(item)

    def pop(self):
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self.stack.pop()

    def peek(self):
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self.stack[-1]

    def is_empty(self):
        return len(self.stack) == 0

    def size(self):
        return len(self.stack)

```

Time Complexity: • PUSH:  $O(1)$  amortized • POP:  $O(1)$  • PEEK:  $O(1)$

Space Complexity:  $O(n)$

## SLIDE 18: STACK - LINKED LIST IMPLEMENTATION

### STACK IMPLEMENTATION USING LINKED LIST

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class StackLinkedList:
    def __init__(self):
        self.top = None
        self._size = 0

    def push(self, item):
        new_node = Node(item)
        new_node.next = self.top
        self.top = new_node
        self._size += 1

    def pop(self):
        if self.is_empty():
            raise IndexError("Stack is empty")

```

```

        popped = self.top.data
        self.top = self.top.next
        self._size -= 1
        return popped

    def peek(self):
        if self.is_empty():
            raise IndexError("Stack is empty")
        return self.top.data

    def is_empty(self):
        return self.top is None

    def size(self):
        return self._size

```

Advantage: Dynamic size, no overflow (until memory full)

## SLIDE 19: STACK APPLICATIONS

### STACK APPLICATIONS

1. Expression Evaluation & Conversion • Infix:  $A + B * C$  • Prefix:  $+ A * B C$  • Postfix:  $A B C *$   
+
2. Parenthesis Matching

```

def is_balanced(expression):
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'}
    for char in expression:
        if char in pairs:
            stack.append(char)
        elif char in pairs.values():
            if not stack or pairs[stack.pop()] != char:
                return False
    return len(stack) == 0

```

3. Undo/Redo Operations • Text editors, Photoshop
4. Browser History (Back Button)
5. Function Call Stack • Recursion management • Local variables storage
6. Tower of Hanoi
7. DFS (Depth First Search)

## SLIDE 20: STACK - INFIX TO POSTFIX CONVERSION

### INFIX TO POSTFIX CONVERSION

Algorithm:

1. Scan expression left to right
2. If operand  $\rightarrow$  output
3. If '('  $\rightarrow$  push to stack
4. If ')'  $\rightarrow$  pop until '('
5. If operator  $\rightarrow$  pop higher/equal precedence, then push

Example:  $A + B * C - D$

| Step | Token | Stack   | Output        |
|------|-------|---------|---------------|
| 1    | A     | empty   | A             |
| 2    | +     | +       | A +           |
| 3    | B     | +       | A + B         |
| 4    | *     | +, *    | A + B *       |
| 5    | C     | +, *    | A + B * C     |
| 6    | -     | +, *, - | A + B * C -   |
| 7    | D     | +, *, - | A + B * C - D |
| End  | pop   | empty   | A + B * C - D |

Result:  $A + B * C - D$

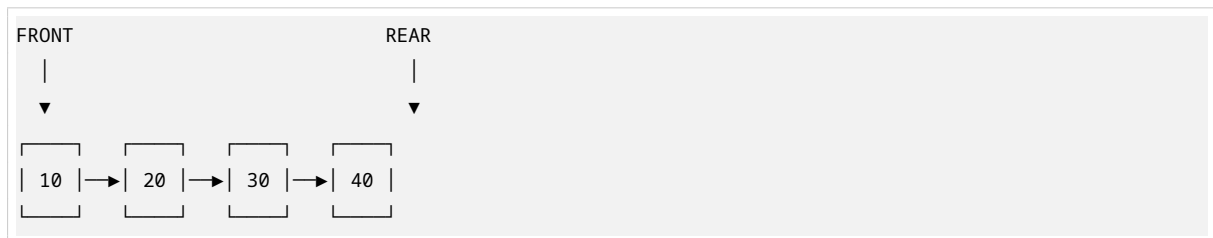
Precedence:  $^ (3) > *, / (2) > +, - (1)$

## SLIDE 21: QUEUE - INTRODUCTION

### QUEUE

Definition: A linear data structure that follows FIFO (First In, First Out) principle.

Visual Representation:



(First In) (Last In) (First Out) (Last Out)

Real-world Analogy: • Ticket counter line • Printer job queue • CPU process scheduling • Call center waiting line

Key Operations:

|            |                                    |
|------------|------------------------------------|
| ENQUEUE    | Add element to rear of queue       |
| DEQUEUE    | Remove element from front of queue |
| FRONT/PEEK | View front element                 |
| isEmpty    | Check if queue is empty            |
| SIZE       | Return number of elements          |

## SLIDE 22: QUEUE - OPERATIONS VISUALIZATION

### QUEUE OPERATIONS - STEP BY STEP

Initial: Empty Queue FRONT = REAR = None

|              |               |       |      |    |    |
|--------------|---------------|-------|------|----|----|
| ENQUEUE(10): | FRONT         | REAR  |      |    |    |
|              |               |       | 10   |    |    |
| ENQUEUE(20): | FRONT         | REAR  |      |    |    |
|              |               |       | 10   | 20 |    |
| ENQUEUE(30): | FRONT         | REAR  |      |    |    |
|              |               |       | 10   | 20 | 30 |
| DEQUEUE()    | → Returns 10: | FRONT | REAR |    |    |
|              |               |       |      | 20 | 30 |

## SLIDE 23: QUEUE - ARRAY IMPLEMENTATION

### QUEUE IMPLEMENTATION USING ARRAY (CIRCULAR)

Problem: Linear array wastes space after dequeue Solution: Circular array with modulo arithmetic

```
class CircularQueue:
    def __init__(self, capacity):
        self.capacity = capacity
        self.queue = [None] * capacity
        self.front = self.rear = -1

    def enqueue(self, item):
        if self.is_full():
            raise OverflowError("Queue is full")
        if self.front == -1:
            self.front = self.rear = 0
        else:
            self.rear = (self.rear + 1) % self.capacity
        self.queue[self.rear] = item

    def dequeue(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        item = self.queue[self.front]
        if self.front == self.rear:
            self.front = self.rear = -1
        else:
            self.front = (self.front + 1) % self.capacity
        return item
```

```
def is_full(self):
    return (self.rear + 1) % self.capacity == self.front

def is_empty(self):
    return self.front == -1
```

## SLIDE 24: QUEUE - LINKED LIST IMPLEMENTATION

### QUEUE IMPLEMENTATION USING LINKED LIST

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class QueueLinkedList:
    def __init__(self):
        self.front = self.rear = None
        self._size = 0

    def enqueue(self, item):
        new_node = Node(item)
        if self.rear is None:
            self.front = self.rear = new_node
        else:
            self.rear.next = new_node
            self.rear = new_node
        self._size += 1

    def dequeue(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        item = self.front.data
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        self._size -= 1
        return item

    def peek(self):
        if self.is_empty():
            raise IndexError("Queue is empty")
        return self.front.data

    def is_empty(self):
```

```
return self.front is None
```

## SLIDE 25: TYPES OF QUEUES

### TYPES OF QUEUES

1. SIMPLE QUEUE (Linear Queue) • Basic FIFO • Wasted space after dequeue
2. CIRCULAR QUEUE • Last position connects to first • Efficient space utilization • Uses modulo for wrap-around
3. PRIORITY QUEUE • Elements have priority values • Higher priority served first • Implementation: Heap or Sorted List

Example: Hospital Emergency Room

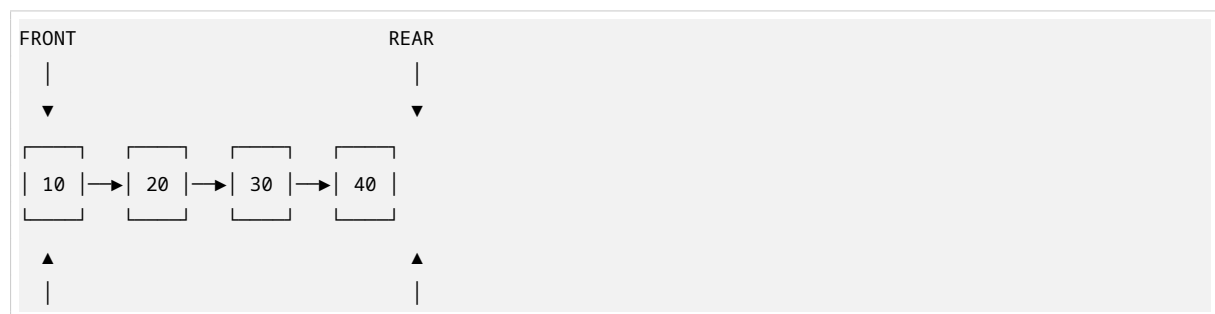
|       | Patient | Priority   |                |
|-------|---------|------------|----------------|
| (Low) | B 1     | (Critical) | ← Served first |
|       | C 2     | (Medium)   |                |
|       | A 3     |            |                |

4. DEQUE (Double-Ended Queue) • Insert/Delete at BOTH ends • Variants:
  - Input Restricted: Insert at one end only
  - Output Restricted: Delete from one end only

## SLIDE 26: DEQUE (DOUBLE ENDED QUEUE)

### DEQUE - DOUBLE ENDED QUEUE

Operations at BOTH ends:



insertFront() insertRear() deleteFront() deleteRear()

```
from collections import deque

# Python's built-in deque
q = deque()
```

```

q.appendleft(10)      # Insert at front
q.append(20)          # Insert at rear
q.appendleft(5)       # Insert at front
# Queue: [5, 10, 20]

q.popleft()           # Remove from front → 5
q.pop()               # Remove from rear → 20

```

Applications: • Sliding window problems • Palindrome checking • Undo operations • Browser history (forward/back)

## SLIDE 27: QUEUE APPLICATIONS

### QUEUE APPLICATIONS

1. CPU Scheduling • Process management • Round-robin scheduling
2. Disk Scheduling • I/O request handling
3. Printer Spooling • Print jobs queued
4. Network Buffers • Packet queuing in routers • Data flow control
5. Breadth-First Search (BFS)

```

def bfs(graph, start):
    visited = set()
    queue = [start]
    visited.add(start)

    while queue:
        node = queue.pop(0)
        print(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

```

6. Cache Implementation (LRU)
7. Message Queues (RabbitMQ, Kafka)
8. Call Center Systems



## SLIDE 28: HASHING - INTRODUCTION

### HASHING

Problem with Arrays/Lists: • Search:  $O(n)$  linear scan • Even sorted:  $O(\log n)$  binary search • Can we do  $O(1)$ ?

Solution: HASHING

Definition: Hashing is a technique to map data of arbitrary size to fixed-size values using a hash function.

Key Concepts:

| Hash Function | Key | Index | (Data) | (0 to m-1) |
|---------------|-----|-------|--------|------------|
|---------------|-----|-------|--------|------------|

Example: Key: “apple”  $\rightarrow$  Hash(“apple”)  $\rightarrow$  3  $\rightarrow$  Store at index 3 Key: “banana”  $\rightarrow$  Hash(“banana”)  $\rightarrow$  7  $\rightarrow$  Store at index 7

Components:

1. Hash Function: Maps key to index
2. Hash Table: Array storing key-value pairs
3. Collision Resolution: Handle same index

## SLIDE 29: HASH FUNCTIONS

### HASH FUNCTIONS

Properties of Good Hash Function: Deterministic: Same key  $\rightarrow$  Same hash Uniform Distribution: Spread evenly Fast Computation:  $O(1)$  time Minimize Collisions

Common Hash Functions:

1. Division Method  $h(k) = k \% m$  where  $m$  = table size (prefer prime)  
Example:  $k=52, m=10 \rightarrow h(52) = 52 \% 10 = 2$
2. Multiplication Method  $h(k) = \text{floor}(m * (k * A \% 1))$  where  $A = 0.6180339887$  (golden ratio)
3. Folding Method Split key into parts, sum them Key: 123456  $\rightarrow 12 + 34 + 56 = 102$
4. String Hashing (djb2)

```
def hash_string(s, table_size):  
    hash_val = 5381  
    for char in s:
```

```

hash_val = ((hash_val << 5) + hash_val) + ord(char)
return hash_val % table_size

```

## SLIDE 30: COLLISION - THE PROBLEM

### COLLISION IN HASHING

What is Collision? When two different keys hash to the same index.

Example: Table Size = 10

$h(\text{"John"}) = 4 \% 10 = 4$   $h(\text{"Jane"}) = 14 \% 10 = 4 \leftarrow \text{COLLISION!}$

|             |   |   |   |   |   |   |   |   |   |                            |
|-------------|---|---|---|---|---|---|---|---|---|----------------------------|
| Hash Table: | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9                          |
| John        |   |   |   |   |   |   |   |   |   | ↑ Jane also wants index 4! |

Collision Resolution Techniques:

1. Separate Chaining (Open Hashing)
2. Open Addressing (Closed Hashing) • Linear Probing • Quadratic Probing • Double Hashing

## SLIDE 31: SEPARATE CHAINING

### SEPARATE CHAINING

Idea: Each bucket is a linked list of elements.

Hash Table (Size 10):

|   |      |   |             |   |      |   |      |   |              |              |
|---|------|---|-------------|---|------|---|------|---|--------------|--------------|
| 0 | None | 1 | None        | 2 | None | 3 | None | 4 | ("John", 25) | ("Jane", 30) |
| 5 | None | 6 | ("Bob", 35) | 7 | None | 8 | None | 9 | None         |              |

```

class HashTableChaining:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self._hash(key)
        for pair in self.table[index]:

```

```

        if pair[0] == key:
            pair[1] = value # Update
            return
        self.table[index].append([key, value])

def search(self, key):
    index = self._hash(key)
    for pair in self.table[index]:
        if pair[0] == key:
            return pair[1]
    return None

```

## SLIDE 32: OPEN ADDRESSING - LINEAR PROBING

## OPEN ADDRESSING - LINEAR PROBING

Idea: If collision, probe next available slot linearly.

Formula:  $h(k, i) = (h'(k) + i) \% m$  where  $i = 0, 1, 2, \dots$  (probe number)

Example: Insert keys 50, 70, 76, 85, 93 Table Size = 10,  $h(k) = k \% 10$

Insert 50:  $h(50) = 0 \rightarrow \text{index } 0$  Insert 70:  $h(70) = 0 \rightarrow \text{index } 0$  occupied probe  $i=1$ :  $(0+1)\%10 = 1 \rightarrow \text{index } 1$  Insert 76:  $h(76) = 6 \rightarrow \text{index } 6$  Insert 85:  $h(85) = 5 \rightarrow \text{index } 5$  Insert 93:  $h(93) = 3 \rightarrow \text{index } 3$

|       |        |    |    |    |    |    |   |   |   |   |   |   |   |   |
|-------|--------|----|----|----|----|----|---|---|---|---|---|---|---|---|
| Final | Table: |    |    |    | 0  | 1  | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|       |        | 50 | 70 | 93 | 85 | 76 |   |   |   |   |   |   |   |   |

### Problem: Primary Clustering

## SLIDE 33: OPEN ADDRESSING - QUADRATIC PROBING

## QUADRATIC PROBING

Idea: Probe with quadratic increments to reduce clustering.

Formula:  $h(k, i) = (h'(k) + c \cdot i + c \cdot i^2) \% m$  Usually:  $h(k, i) = (h'(k) + i^2) \% m$

Example: Insert keys 50, 70, 76, 93 Table Size = 10,  $h(k) = k \% 10$

Insert 50:  $h(50) = 0 \rightarrow \text{index } 0$  Insert 70:  $h(70) = 0 \rightarrow \text{index } 0$  i=1:  $(0 + 1^2)\%10 = 1 \rightarrow \text{index } 1$  Insert 76:  $h(76) = 6 \rightarrow \text{index } 6$  Insert 93:  $h(93) = 3 \rightarrow \text{index } 3$

Search 70:  $h(70) = 0 \rightarrow \text{not } 70 \text{ i}=1: (0 + 1)\%10 = 1 \rightarrow \text{found } 70$

Advantage: Reduces primary clustering Problem: Secondary clustering (same probe sequence)

## SLIDE 34: DOUBLE HASHING

### DOUBLE HASHING

Idea: Use TWO hash functions for probe sequence.

Formula:  $h(k, i) = (h(k) + i * h_2(k)) \% m$

Requirements: •  $h(k) \neq 0$  for any  $k$  •  $h(k)$  and  $m$  should be relatively prime

Example:  $h(k) = k \% 10$   $h_2(k) = 7 - (k \% 7)$

Insert 50:  $h(50)=0 \rightarrow$  index 0 Insert 70:  $h(70)=0 \rightarrow$  index 0  $h_2(70) = 7 - (70\%7) = 7$   $i=1: (0 + 1*7)\%10 = 7 \rightarrow$  index 7 Insert 76:  $h(76)=6 \rightarrow$  index 6

|        |    |   |    |    |   |   |   |   |   |   |   |
|--------|----|---|----|----|---|---|---|---|---|---|---|
| Table: |    | 0 | 1  | 2  | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|        | 50 |   |    |    |   |   |   |   |   |   |   |
|        |    |   | 76 | 70 |   |   |   |   |   |   |   |

Advantage: Eliminates clustering entirely

## SLIDE 35: HASH TABLE COMPLEXITY

### HASH TABLE TIME COMPLEXITY

|        | Operation |          | Average Case | Worst Case |          | Space  |        |          |  |  |
|--------|-----------|----------|--------------|------------|----------|--------|--------|----------|--|--|
| Insert | $O(1)$    | $O(n)^*$ | Delete       | $O(1)$     | $O(n)^*$ | Search | $O(1)$ | $O(n)^*$ |  |  |

- Worst case: All keys hash to same index (degenerate to linked list search)

Load Factor  $(\lambda) = n / m$   $n$  = number of elements  $m$  = table size

Guidelines: • Keep load factor  $< 0.75$  • Rehash when load factor exceeds threshold • Choose table size as prime number

| Comparison:       | Operation |             |        | Array  | BST    | HashTable   |
|-------------------|-----------|-------------|--------|--------|--------|-------------|
| Search            | $O(n)$    | $O(\log n)$ | $O(1)$ | Insert | $O(1)$ | $O(\log n)$ |
|                   |           |             |        | Delete | $O(n)$ | $O(\log n)$ |
| Ordered Traversal | $O(n)$    | $O(n)$      | $O(n)$ |        |        |             |

## SLIDE 36: HASHING APPLICATIONS

### HASHING APPLICATIONS

1. Database Indexing • Fast record lookup • Hash indexes in SQL
2. Caching • LRU Cache (Hash + Doubly Linked List) • Memcached, Redis
3. Password Storage • Store hash, not plain password • SHA-256, bcrypt, Argon2
4. Cryptocurrency • Bitcoin: SHA-256 hashing • Block verification
5. Compiler Design • Symbol tables • Keyword identification
6. Data Deduplication • File systems • Backup systems
7. Load Balancing • Consistent hashing • Distributed systems
8. Digital Signatures • Document integrity verification

## SLIDE 37: PYTHON DICTIONARY - HASH TABLE IN ACTION

### PYTHON DICTIONARY - HASH TABLE IMPLEMENTATION

Python's dict is a highly optimized hash table.

```
# Creating a dictionary
student = {
    "name": "Alice",
    "age": 20,
    "grade": "A"
}

# Operations
student["major"] = "CS"      # Insert: O(1)
print(student["name"])      # Search: O(1)
del student["age"]          # Delete: O(1)

# Iteration
for key, value in student.items():
    print(f"{key}: {value}")

# Built-in methods
student.keys()              # All keys
```

```

student.values()    # All values
student.get("gpa", 0) # Safe access with default

# Dictionary comprehension
squares = {x: x**2 for x in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

```

Python 3.7+: Dictionaries maintain insertion order!

## SLIDE 38: COMPLETE CODE - ALL DATA STRUCTURES

### COMPLETE IMPLEMENTATION SUMMARY

Here's a quick reference of all structures:

```

# ===== LINKED LIST =====
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self): self.head = None
    def insert(self, data): ...
    def delete(self, key): ...
    def search(self, key): ...

# ===== STACK =====
class Stack:
    def __init__(self): self.items = []
    def push(self, item): self.items.append(item)
    def pop(self): return self.items.pop()
    def peek(self): return self.items[-1]
    def is_empty(self): return len(self.items) == 0

# ===== QUEUE =====
from collections import deque
class Queue:
    def __init__(self): self.items = deque()
    def enqueue(self, item): self.items.append(item)
    def dequeue(self): return self.items.popleft()
    def front(self): return self.items[0]

# ===== HASH TABLE =====
class HashTable:
    def __init__(self, size=100):
        self.size = size

```

```

        self.table = [[] for _ in range(size)]
def insert(self, key, value): ...
def search(self, key): ...
def delete(self, key): ...

```

## SLIDE 39: COMPARISON TABLE

### DATA STRUCTURES COMPARISON

|            |          |            |           |               |            | Feature     | Linked List | Stack   | Queue | Hash Table |         |
|------------|----------|------------|-----------|---------------|------------|-------------|-------------|---------|-------|------------|---------|
|            |          |            |           |               |            | Order       | Sequential  | LIFO    | FIFO  | Unordered  | Access  |
| Sequential | Top only | Front/Rear | Key-based | Insert (best) | O(1)       | O(1)        | O(1)        | O(1)    | O(1)  | O(1)       | Delete  |
| (best)     | O(1)     | O(1)       | O(1)      | O(1)          | Search     | O(n)        | O(n)        | O(n)    | O(1)  | Memory     | Dynamic |
| Dynamic    | Fixed*   | Use Case   | Dynamic   | Undo/Redo     | Scheduling | Fast Lookup | memory      | Parsing |       |            |         |
| BFS        | Caching  |            |           |               |            |             |             |         |       |            |         |

- Can be resized dynamically

When to Use What? • Need fast lookup → Hash Table • Need order preservation → Linked List • Need undo functionality → Stack • Need fair processing → Queue • Need both ends access → Deque

## SLIDE 40: PRACTICE PROBLEMS

### PRACTICE PROBLEMS

LINKED LIST: Reverse a linked list (iterative & recursive) Detect cycle in linked list (Floyd's algorithm)

Find middle element in one pass Merge two sorted linked lists Remove duplicates from sorted list

STACK: Implement two stacks in one array Sort a stack using another stack Next Greater Element

Evaluate postfix expression Design a stack with getMin() in O(1)

QUEUE: Implement stack using queues Implement queue using stacks Circular tour problem First non-repeating character in stream Sliding window maximum

HASHING: Two Sum problem Group Anagrams Longest Consecutive Sequence LRU Cache implementation Find first repeating element

## SLIDE 41: KEY TAKEAWAYS

### KEY TAKEAWAYS

Linked List • Dynamic size, efficient insertion/deletion • Types: Singly, Doubly, Circular • Trade-off: No random access

Stack • LIFO principle • Perfect for undo, parsing, recursion • Array or linked list implementation

Queue • FIFO principle • Essential for BFS, scheduling, buffering • Variants: Simple, Circular, Priority, Deque

Hashing •  $O(1)$  average case for search/insert/delete • Collision resolution: Chaining, Open Addressing • Foundation of dictionaries, caches, databases

Remember: “Choose the right data structure for the problem, and the algorithm becomes obvious.”

## SLIDE 42: REFERENCES & RESOURCES

### REFERENCES & RESOURCES

Books: • “Introduction to Algorithms” - Cormen, Leiserson, Rivest, Stein • “Data Structures and Algorithms in Python” - Goodrich, Tamassia • “The Algorithm Design Manual” - Steven Skiena

Online Platforms: • LeetCode (leetcode.com) • HackerRank (hackerrank.com) • GeeksforGeeks (geeksforgeeks.org) • Visualgo (visualgo.net) - Interactive visualizations

YouTube Channels: • Abdul Bari (Data Structures) • mycodeschool • CS Dojo

Tools: • Python Tutor (pythontutor.com) • Data Structure Visualizer

## SLIDE 43: Q&A

### QUESTIONS & ANSWERS

Thank You!

Questions?

 [Your Email]  [Your Website/GitHub]



“The code you write makes you a programmer. The data structures you choose make you a computer scientist.”

**END OF PRESENTATION**